



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

LAB ASSIGNMENT-2

Submitted by:

Kaushtubh Kumar

1024150191

2023

1. Create a structure in C++ containing the details of Students as details below and a main function to execute the structure.

Data Members (properties):

- Name
- Roll No
- Degree
- Hostel
- Current CGPA

Member Functions (behavior):

1. addDetails()
2. updateDetails()
3. updateCGPA()
4. updateHostel()
5. displayDetails()

CODE
<pre>#include <iostream> using namespace std; struct Students</pre>

```

{
    string Name;
    int rollno;
    string degree;
    string hostel;
    float currentCGPA;

public:
    void add_details(){
        cout<<"enter name :";
        cin>>Name;

        cout<<"enter rollno :";
        cin>>rollno;

        cout<<"enter degree :";
        cin>>degree;

        cout<<"enter hostel :";
        cin>>hostel;

        cout<<"enter currentCGPA :";
        cin>>currentCGPA;
    }

    void display_details(){
        cout<<"entered name\t"<<Name<<endl;
        cout<<"entered rollno\t"<<rollno<<endl;
        cout<<"entered degree\t"<<degree<<endl;
        cout<<"entered hostel\t"<<hostel<<endl;
        cout<<"entered cgpa\t"<<currentCGPA<<endl;
    }

    void update_details(){
        cout<<"ENTER NEW NAME :";
        cin>>Name;

        cout<<"ENTER NEW ROLLNO :";
        cin>>rollno;

        cout<<"ENTER NEW DEGREE :";
        cin>>degree;

        cout<<"ENTER NEW HOSTEL :";
        cin>>hostel;

        cout<<"ENTER NEW CGPA :";
        cin>>currentCGPA;
    }

    void update_cgpa(){
        cout<<"UPDATE CGPA : " ;
        cin>>currentCGPA;
    }
}

```

```

    }

    void update_hostel(){
        cout<<"UPDATE HOSTEL :";
        cin>>hostel;
    }

};

int main(){
    int n;
    cout<<"ENTER NO OF STUDENTS FOR ENTRY: ";
    cin>>n;
    Students s[n];

    for(int i = 0; i < n; i++){
        cout << "\n--- ENTER DETAILS OF STUDENTS " << i+1 << " ---\n";
        s[i].add_details();
    }

    for(int i = 0; i < n; i++){
        cout << "\n DETAILS OF STUDENTS " << i+1 << " ---\n";
        s[i].display_details();
    }

    Students x;
    x.add_details();
    x.update_details();
    x.update_cgpa();
    x.update_hostel();
    x.display_details();

    return 0;
}

```

OUTPUT

```

PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
oops.cpp -o oops } ; if ($?) { .\oops }
ENTER NO OF STUDENTS FOR ENTRY: 2

--- ENTER DETAILS OF STUDENTS 1 ---
enter name :scot
enter rollno :1
enter degree :CS
enter hostel :A
enter currentCGPA :5.5

--- ENTER DETAILS OF STUDENTS 2 ---
enter name :James
enter rollno :2
enter degree :AI
enter hostel :B
enter currentCGPA :8.3

DETAILS OF STUDENTS 1 ---
entered name      scot
entered rollno    1
entered degree    CS
entered hostel    A
entered cgpa      5.5

DETAILS OF STUDENTS 2 ---
entered name      James
entered rollno    2
entered degree    AI
entered hostel    B
entered cgpa      8.3
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>

```

```

PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques> cd "c
oops.cpp -o oops } ; if ($?) { .\oops }
enter name :rashu
enter rollno :123
enter degree :enc
enter hostel :K
enter currentCGPA :6.9
ENTER NEW NAME :costubh
ENTER NEW ROLLNO :123
ENTER NEW DEGREE :coe
ENTER NEW HOSTEL :M
ENTER NEW CGPA :7.9
UPDATE CGPA :8.1
UPDATE HOSTEL :X
entered name      costubh
entered rollno    123
entered degree    coe
entered hostel    X
entered cgpa      8.1
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>

```

Q2.Differentiate between private and public access / scope.
Perform Question No. 1 using class instead of structure, having the data members private and some member functions in private scope and some in public scope

Public: Members are accessible from outside the class (e.g., in main).

Private: Members are only accessible within the class itself. They are hidden from the outside world to ensure data security (Encapsulation).

CODE

```
#include <iostream>
#include <string>
using namespace std;

class StudentClass {
private:
    string Name;
    int RollNo;
    string Degree;
    string Hostel;
    float CurrentCGPA;

    void access_pvt() {
        cout<<endl;
        cout << "Accessed a private function\n";
    }

public:
    void addDetails() {
        cout << "Enter Name: "; cin >> Name;
        cout << "Enter Roll No: "; cin >> RollNo;
        cout << "Enter Degree: "; cin >> Degree;
        cout << "Enter Hostel: "; cin >> Hostel;
        cout << "Enter CGPA: "; cin >> CurrentCGPA;

        access_pvt();
    }

    void update_details(){
        cout<<"ENTER NEW NAME :";
        cin>>Name;

        cout<<"ENTER NEW ROLLNO :";
        cin>>RollNo;
```

```

        cout<<"ENTER NEW DEGREE :";
        cin>>Degree;

        cout<<"ENTER NEW HOSTEL :";
        cin>>Hostel;

        cout<<"ENTER NEW CGPA :";
        cin>>CurrentCGPA;
    }

    void updateCGPA() {
        cout << "Enter Updated CGPA: "; cin >> CurrentCGPA;
    }

    void updateHostel() {
        cout << "Enter Updated Hostel: "; cin >> Hostel;
    }

    void displaydetails() {
        cout << "\nStudent Details (Class)\n";
        cout << "Name: " << Name << "\nRoll No: " << RollNo
            << "\nDegree: " << Degree << "\nHostel: " << Hostel
            << "\nCGPA: " << CurrentCGPA << endl;
    }
};

int main() {
    StudentClass s1;
    s1.addDetails();
    s1.displaydetails();
    return 0;
}

```

OUTPUT

```
oops.cpp -o oops } ; if ($?) { .\oops }
```

```
Enter Name: A
```

```
Enter Roll No: 1
```

```
Enter Degree: EEE
```

```
Enter Hostel: S
```

```
Enter CGPA: 8.6
```

```
Accessed a private function
```

```
Student Details (Class)
```

```
Name: A
```

```
Roll No: 1
```

```
Degree: EEE
```

```
Hostel: S
```

```
CGPA: 8.6
```

```
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```

Q3.Create a code snippet that illustrates the following:

Calling of private member functions inside public member function.

CODE

```
#include <iostream>
using namespace std;

class Alpha {
private:
    void call_privateFunction() {
        cout << "Inside Private function" << endl;
    }

public:
    void publicFunction() {
        cout << "Inside Public function" << endl;
        call_privateFunction();
    }
};

int main() {
    Alpha x;
    x.publicFunction();
    return 0;
}
```

```
}
```

OUTPUT

```
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques> g++ oops.cpp -o oops } ; if ($?) { .\oops }
Inside Public function
Inside Pruvate function
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```

Q4. Define a class Rectangle with variables width and height of integer type along with the following:

(a) void getdata() to initialize object values

(b) void calculateArea() to calculate and display the area

CODE

```
#include <iostream>
using namespace std;
#include <vector>

class rectangle{
    private:
        float length;
        float breadth;

    public:
        void get_data(){
            cout<<"ENTER LENGTH :";
            cin>>length;
            cout<<"ENTER BREADTH :";
            cin>>breadth;
            cout<<"L ="<<length<<" , B ="<<breadth<<endl;
        }
}
```

```
float calculate_area(){
    return length*breadth;
}
};

int main(){
rectangle r;
r.get_data();
cout<<"AREA = "<<r.calculate_area()<<endl;
}
```

OUTPUT

```
oops.cpp -o oops } ; if ($?) { .\oops }
ENTER LENGTH :45
ENTER BREADTH :60
L =45, B =60
AREA = 2700
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\
```

Q5. Define a class Complex with variables real and imaginary along with the following:

- (a) void setComplex(float, float) to initialize object values**
- (b) void displayComplex() to show the complex number**
- (c) Pass and return objects to calculate sum of two complex numbers. Display the sum.**

CODE

```
#include <iostream>
using namespace std;

class Complex {
    float real;
    float imaginary;

public:
    void setComplex(float r, float i) {
        real = r;
        imaginary = i;
    }

    void displayComplex() {
        cout << real << " + " << imaginary << "i" << endl;
    }

    Complex sum(Complex c2) {
        Complex temp;
        temp.real = real + c2.real;
        temp.imaginary = imaginary + c2.imaginary;
        return temp;
    }
};

int main() {
    Complex c1, c2, c3;

    c1.setComplex(3.5, 2.5);
    c2.setComplex(1.5, 4.5);

    cout << "C1: "; c1.displayComplex();
    cout << "C2: "; c2.displayComplex();

    c3 = c1.sum(c2);

    cout << "Sum: "; c3.displayComplex();
    return 0;
}
```

OUTPUT

```
tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) {
C1: 3.5 + 2.5i
C2: 1.5 + 4.5i
Sum: 5 + 7i
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```

Q6.Implement scope resolution operator :: for the following uses:

- (a) Class functions defined outside the class**
- (b) Access a global variable with the same name as a local variable**
- (c) Access static variables**
- (d) Use inbuilt libraries (cin, cout) with scope resolution operator**

CODE

```
#include <iostream>

int num = 100;

class Scopes {
    static int staticVar;
public:
    void show();
    static void showStatic() {
        std::cout << "Static Variable: " << staticVar << std::endl;
    }
};

int Scopes::staticVar = 50;

void Scopes::show() {
    std::cout << "Function defined outside class." << std::endl;
}

int main() {
    int num = 20;

    std::cout << "Local num: " << num << std::endl;

    std::cout << "Global num: " << ::num << std::endl;
```

```

    Scopes inst;
    inst.show();

    Scopes::showStatic();

    return 0;
}

```

OUTPUT

```

oops.cpp -o oops } ; if ($?) { .\oops }
● Local num: 20
  Global num: 100
  Function defined outside class.
  Static Variable: 50
○ PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>

```

Q7.Implement *namespace* in a program to illustrate the use of same name variables and functions in different sections/libraries of the code.

CODE

```

#include <iostream>
using namespace std;

namespace NamespaceA {
    int val = 10;
    void show() {
        cout << "Inside Namespace A, val = " << val << endl;
    }
}

namespace NamespaceB {
    int val = 20;
    void show() {
        cout << "Inside Namespace B, val = " << val << endl;
    }
}

```

```
    }  
}  
  
int main() {  
    NamespaceA::show();  
    cout << "Direct access A: " << NamespaceA::val << endl;  
  
    NamespaceB::show();  
    cout << "Direct access B: " << NamespaceB::val << endl;  
  
    return 0;  
}
```

OUTPUT

```
tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) {  
Inside Namespace A, val = 10  
Direct access A: 10  
Inside Namespace B, val = 20  
Direct access B: 20  
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```